

ROS2 day2 hw2 레포트

로봇 예비단원 주호진

1. 과제

-QT GUI ros패키지로 turtlesim조종

1일차 3번 과제 기능 전부 포함 (WASD키보드 입력/삼각형, 사각형, 원 출력/굵기, 색상 다르게 설정)

QPushButton사용 cmd_vel값 gui출력

2. 코드

- 3번 과제 기능 가져오기

```

import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist
from turtlesim.srv import SetPen
import sys
import select
import tty
import termios
import threading

class TurtlesimControl(Node):
    > def __init__(self): ...
    > def keyboard_listener_loop(self): ...
    > def control_loop_callback(self): ...
    > def set_pen(self, r, g, b, width, off=0): ...
    > def circle(self): ...
    > def rectangle(self): ...
    > def triangle(self): ...
    > def pentagon(self): ...
    > def main(args=None): ...

if __name__ == '__main__':
    main()

```

Cpp 패키지를 사용하는 것이 조건이었기 때문에 py 패키지에서 만들었던 기능을 cpp로 이식했다.

```

#include <geometry_msgs/msg/twist.hpp>
#include <rclcpp/rclcpp.hpp>
#include <turtlesim/srv/set_pen.hpp>
from geometry_msgs.msg import Twist
from turtlesim.srv import SetPen

```

```
private:
    void keyboardListenerLoop();
    void controlLoop();

    void setPen(std::uint8_t red, std::uint8_t green, std::uint8_t blue, std::uint8_t width, std::uint8_t off = 0);

    void circle();
    void triangle();
    void rectangle();
    void pentagon();

    std::shared_ptr<rclcpp::Node> node;
    rclcpp::Publisher<geometry_msgs::msg::Twist> publisher;

    rclcpp::Client<turtlesim::srv::SetPen> penClient;
    std::atomic<bool> isRunning{true};
    std::atomic<char> currentMode{'\0'};
    std::atomic<int> i{0};
    std::thread keyboardThread;
```

헤더 파일에 py에서 사용했던 함수와 변수, 라이브러리를 가져와주었다.

```
def circle(self):
    if self.i == 0:
        self.set_pen(0, 255, 0, 4)

    msg = Twist()
    msg.linear.x = 2.0 # 전진 속도
    msg.angular.z = 2.0 # 회전 속도
    self.publisher.publish(msg)
    self.get_logger().info(f'Publishing: "linear.x={msg.linear.x}"')

    if self.i > 4:
        stop_msg = Twist()
        self.publisher.publish(stop_msg)
        self.current_mode = None
        self.i = 0
    self.i += 1

void QNode::circle()
{
    if (i == 0)
        setPen(0, 255, 0, 4);

    geometry_msgs::msg::Twist msg;
    msg.linear.x = 2.0;
    msg.angular.z = 2.0;
    publisher->publish(msg);
    if (i > 4)
    {
        publisher->publish(geometry_msgs::msg::Twist());
        currentMode = '\0'; // null char
        i = 0;
    }
    i += 1;
}
```

(circle 이하 도형 그리는 함수 로직 동일(triangle, rectangle, pentagon))

```
def keyboard_listener_loop(self): ...
```

```

void QNode::keyboardListenerLoop()
{
    if (!isatty(STDIN_FILENO)) // 터미널인지
        return;

    termios settings{};
    if (tcgetattr(STDIN_FILENO, &settings) != 0) // 터미널 설정 받아오기
        return;

    termios rawSettings = settings;
    rawSettings.c_lflag &= static_cast<unsigned long>(~(ICANON | ECHO)); // 즉시 입력 받기, 문자 출력X
    rawSettings.c_cc[VMIN] = 0;
    rawSettings.c_cc[VTIME] = 0;
    tcsetattr(STDIN_FILENO, TCSADRAIN, &rawSettings);

    while (isRunning && rclcpp::ok())
    {
        fd_set inputSet;
        FD_ZERO(&inputSet);
        FD_SET(STDIN_FILENO, &inputSet);
        timeval timeout{0, 100000}; // s, us = 0.1s
        if (select(STDIN_FILENO + 1, &inputSet, nullptr, nullptr, &timeout) > 0)
        {
            // 입력이 있는 경우
            char key = '\0'; // null
            // 키 입력 && 그리는 중 X && wasd중 하나인 경우
            if (read(STDIN_FILENO, &key, 1) == 1 && currentMode == '\0' &&
                (key == 'w' || key == 'a' || key == 's' || key == 'd'))
            {
                currentMode = key;
                i = 0;
            }
        }
    }
}

```

소스파일에는 선언된 함수들을 옮겨주는 작업과, ros2 qt 패키지 기본으로 설정된 qnode 형식과 연결되는 부분을 수정했다.

```

this->start(); // start -> run() 실행
}

```

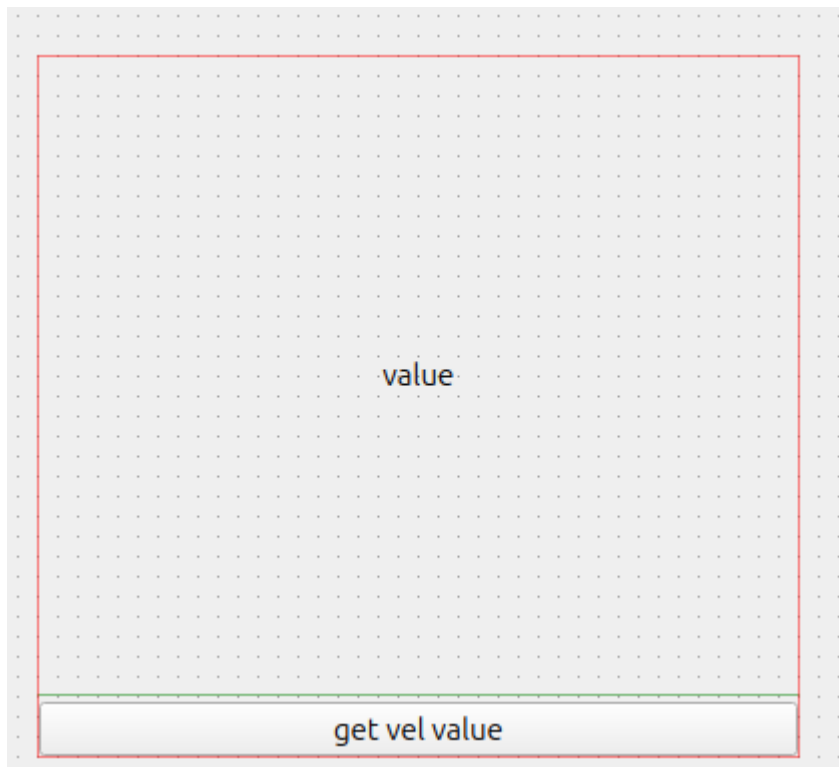
```

void QNode::run()
{
    rclcpp::WallRate loop_rate(20);

    auto lastControl = std::chrono::steady_clock::now();
    while (rclcpp::ok()) // ok(): ros2 실행 여부, shutdown 시 false
    {
        rclcpp::spin_some(node); // ros2에서 처리해야 할 콜백 처리
        const auto now = std::chrono::steady_clock::now();
        if (now - lastControl >= std::chrono::seconds(1)) // loop 1s마다 실행
        {
            controlLoop();
            lastControl = now;
        }
        loop_rate.sleep(); // 과연산 방지
    }
}

```

- pushBtn event(signal)와 vel값 연결



.ui상에서는 간단하게 아래 버튼을 누르면 위 label에 vel값이 뜨도록 위젯 2개를 배치했다.

- Vel값 읽어오기

Vel값을 받아오기 위해서는 turtlesim에서 pub되는 /cmd_vel 토픽의 vel값을 subscribe해서 ui와 연동해주면 된다. 앞서 터틀봇 과제에서 확인한 결과는 turtlesim에서 속도의 x값과 각속도의 z값만 사용하기 때문에 두개의 값을 가져오도록 설정한다.

```
#include <geometry_msgs/msg/twist.hpp>
#include <rclcpp/rclcpp.hpp>
```

Turtle을 wasd로 제어할 때 해당 msg를 이미 include하였으므로 추가로 include할 메시지는 없다.

```
void velCallback(const geometry_msgs::msg::Twist::SharedPtr msg);
rclcpp::Subscription<geometry_msgs::msg::Twist>::SharedPtr velSub;
```

```
public:
```

```
std::atomic<float> linearXValue{0.0};
std::atomic<float> angularZValue{0.0};
```

헤더에 속도 x와 각속도 z를 받는 콜백 함수를 선언하고 그 값을 저장할 float 변수를 선언한다.

```
velSub = node->create_subscription<geometry_msgs::msg::Twist>(
    "/turtle1/cmd_vel", 10, std::bind(&QNode::velCallback, this, std::placeholders::_1));
```

```
void QNode::velCallback(const geometry_msgs::msg::Twist::SharedPtr msg)
{
    linearXValue.store(msg->linear.x);
    angularZValue.store(msg->angular.z);
}
```

subcribe되면 헤더에 선언했던 변수에 lin X와 ang Z값을 저장한다.

- UI 연결

이 값을 UI에 띄우기 위해서 Qt 파일(mainwindow) 헤더에 pushBtn 슬롯을 선언해준다.

```
private slots:
    void on_pushButton_clicked();
```

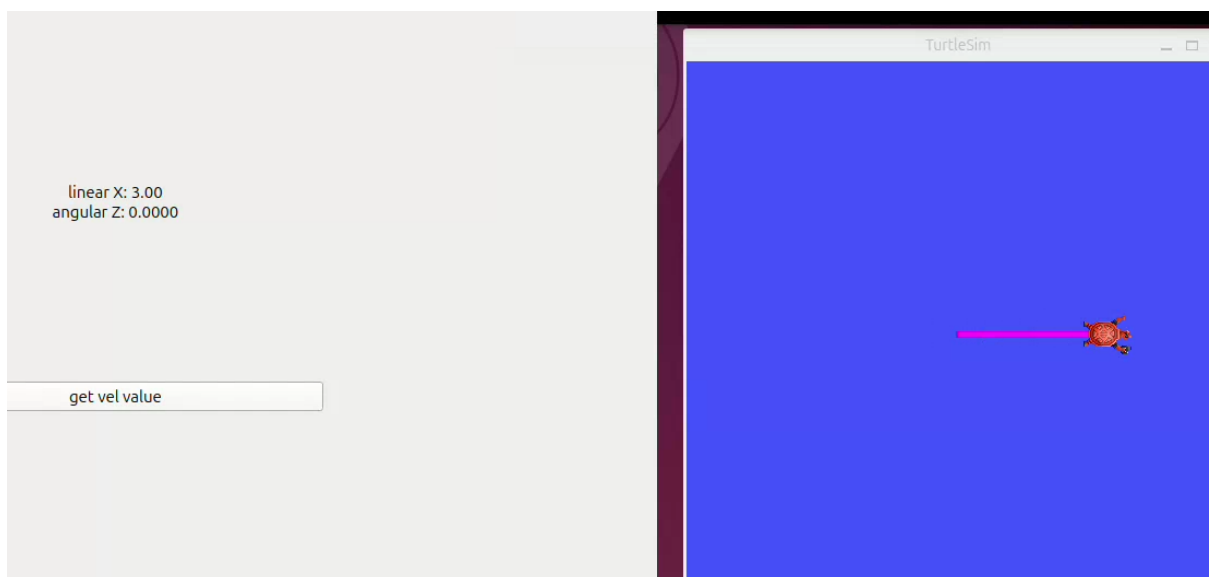
해당 슬롯에서 label에 vel값(속도 x, 각속 z)을 띄우도록 연결해준다.

```
void MainWindow::on_pushButton_clicked()
{
    const QString velText =
        QString("linear X: %1\nangular Z: %2").arg(qnode->linearXValue, 0, 'f', 2).arg(qnode->angularZValue, 0, 'f', 4);
    ui->label->setText(velText);
}
```

3. 작동 영상

작동 영상

- 직진



- 회전 (삼각형 120도 $\approx 2.0940\text{rad}$)

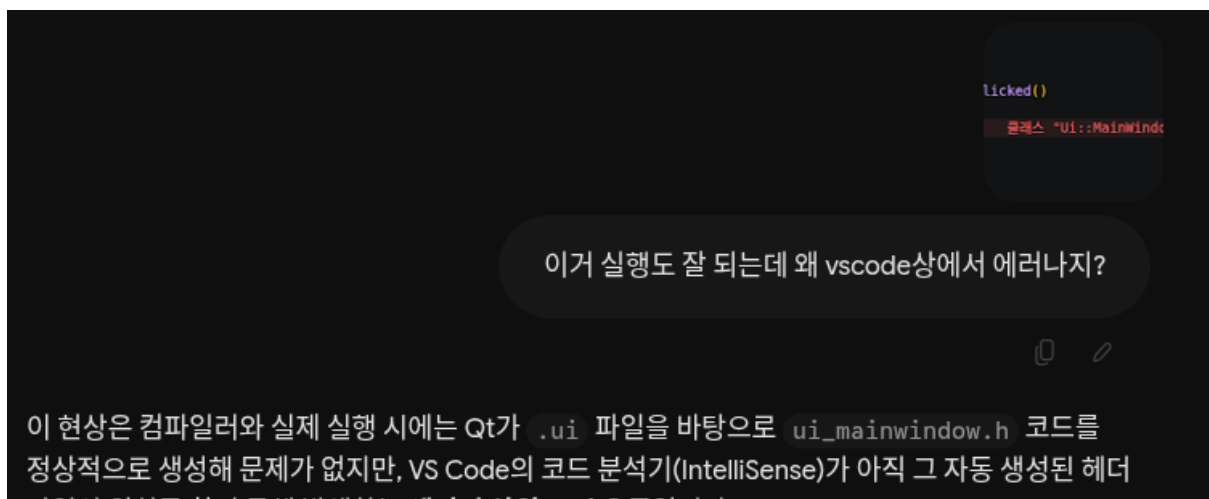


4. AI 보고서

- ROS2 qt 패키지 설정 관련

<https://share.gemini.google/o9mktxyDXSmX>

<https://share.gemini.google/x2tVH7jWAqZE>



ros2 qt 패키지를 vscode 환경에 세팅하는 것과 ros2 qt 패키지에서 빌드가 안되어서 cmakeLists과 생성 스크립트 등을 수정하였다.

- py 패키지

<https://share.gemini.google/THufOq2LxdcC>

이걸 cpp 패키지로 이식해줄건데 CMakeLists랑 xml은 geometry_msgs랑 turtlesim 추가해주면 되나?

네, 기본적으로 geometry_msgs 와 turtlesim 외에도 C++용 클라이언트 라이브러리인 rclcpp 가 반드시 필요합니다. package.xml 과 CMakeLists.txt 에 아래 항목들을 추가해야 합니다.

keyboard_listener_loop 애는 c++로 구현하려면 어떻게 해

C++에서는 POSIX 시스템 콜(<termios.h> , <unistd.h> , <sys/select.h>)과 표준 스레드(<thread> , <mutex>)를 사용해 구현합니다.

day1의 turtle봇 제어 조건을 cpp패키지로 이식해주기 위해 질문하였다.

```
main_window.hpp M X main_window.cpp M X turtlesim_control_node.py X
> day1 > hw3 > day1_hw3_ros2_py_package > day1_hw3_ros2_py_package > turtlesim
1 import rclpy
2 from rclpy.node import Node
3 from geometry_msgs.msg import Twist
4 from turtlesim.srv import SetPen
5 import sys
6 import select
7 import tty
8 import termios
9 import threading
10
11 class TurtlesimControl(Node):
12     def __init__(self):
13     def keyboard_listener_loop(self):
14     def control_loop_callback(self):
15
16     def set_pen(self, r, g, b, width, off=0):
17
18     def circle(self):
19
20     def rectangle(self):
21
22     def triangle(self):
23
24     def pentagon(self):
25
26 def main(args=None):
27
28 if __name__ == '__main__':
29     main()
```

Day1_hw3의 turtlesim_control_node.py 파일을 cpp로 바꾸는 과정에서 키보드 구현을 위해 질문했다. ros2 qt 패키지로 생성시 qThread라는 형식으로 qnode가 기본 파일로 생성되어 있어 해당 구조에 맞게 이식했다. 처음에는 day1에서 구현했던 것처럼 노드에 main을 넣었는데 해당 패키지에 main.cpp가 있어 충돌이 일어나게 되어 변경하였다.